SETUP — rodar este projeto em uma máquina nova

Guia completo para deixar o **Tech Challenge Fase 04** funcionando do zero. Serve para Windows, macOS e Linux.

> **Se você é um assistente de IA lendo este arquivo:** este é um monorepo com `backend/` (API Node/Express/MongoDB) e `mobile/` (app Expo / React Native). O usuário quer a aplicação rodando. Siga as etapas 1 a 6 na ordem, rodando os comandos de verificação antes de instalar qualquer coisa — boa parte já pode estar instalada. A etapa 4 tem três caminhos alternativos para o banco; escolha um só. Não suponha que o `.env` existe: ele é ignorado pelo git e precisa ser criado a partir do `.env.example`.

Resumo em 60 segundos

Se a máquina já tem Node 22.13+, Docker e o emulador Android configurados:

```
git clone <url-do-repositorio>
cd tech-challenge-fase4

cp backend/.env.example backend/.env

cd backend && npm ci && cd ..
cd mobile  && npm ci && cd ..

docker compose up -d --build
docker compose exec api npm run seed

cd mobile && npm run android
```

Login: `professor@postech.com` / `postech2026`.

> No PowerShell, troque `cp` por `Copy-Item backend\.env.example backend\.env`. Os demais comandos são iguais.

Se algo aí falhou, siga o guia completo abaixo.

---

1 · Pré-requisitos

1.1 Node.js — obrigatório, versão exata importa

```
node -v
```

Resultado	O que fazer
`v22.13.x` ou superior (22.13+, 24.3+, 26+)	Pronto, siga em frente
`v18.x`, `v20.x`, `v22.0` – `v22.12`	**Precisa atualizar.** O Expo SDK 58 e o React Native 0.88 declaram `engines: ^22.13.0 || ^24.3.0 || >=26`. Em versão menor o Metro falha ou quebra em runtime
comando não encontrado	Instale

Instalar / atualizar:

```
# Windows
winget install -e --id OpenJS.NodeJS.LTS
```

```
# macOS
brew install node@22
```

```
# Linux (Ubuntu/Debian)
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash - && sudo apt install -y nodejs
```

Se você usa **nvm** (recomendado quando há outros projetos na máquina):

```
nvm install 22.13.0
nvm use 22.13.0
```

> O backend sozinho roda em Node 18. Quem exige 22.13+ é o app mobile. Como o repositório é um monorepo, use 22.13+ para os dois e não pense mais nisso.

1.2 Git

```
git --version
```

```
# Windows
winget install -e --id Git.Git
```

1.3 O que não é obrigatório

Ferramenta	Precisa?
Docker Desktop	Só se você escolher o caminho A do banco (etapa 4)
MongoDB instalado	Só se escolher o caminho C
Android Studio / SDK	Só para rodar no emulador. Celular físico e navegador dispensam
JDK / Java	Não. O Expo em modo gerenciado não compila Android localmente
Xcode	Só para simulador iOS, e só em macOS

---

2 · Clonar e instalar dependências

```
git clone <url-do-repositorio>
cd tech-challenge-fase4
```

`node_modules` **não** está no repositório. Instale nos dois projetos:

```
cd backend && npm ci && cd ..
cd mobile  && npm ci && cd ..
```

Use `npm ci` (respeita o `package-lock.json`, que está versionado). Se o `npm ci` reclamar de lock desatualizado, caia para `npm install`.

A primeira instalação do backend baixa o binário do MongoDB usado nos testes (~100 MB). É normal demorar alguns minutos.

---

3 · Variáveis de ambiente

3.1 Backend — obrigatório

O arquivo `.env` é ignorado pelo git, então ele **não vem no clone**. Crie:

```
cp backend/.env.example backend/.env
```

```
# Windows (PowerShell)
Copy-Item backend\.env.example backend\.env
```

Conteúdo e o que ajustar:

```
PORT=3000
MONGODB_URI=mongodb://mongodb:27017/blogdb
NODE_ENV=development
JWT_SECRET=troque-este-segredo-em-producao
JWT_EXPIRES_IN=7d
SEED_PROFESSOR_EMAIL=professor@postech.com
SEED_PROFESSOR_SENHA=postech2026
```

Variável	Quando mexer
`MONGODB_URI`	`mongodb://mongodb:27017/blogdb` funciona **dentro do Docker Compose** (`mongodb` é o nome do serviço). Para MongoDB instalado na máquina, troque para `mongodb://localhost:27017/blogdb`
`JWT_SECRET`	Troque por qualquer string longa. **Se mudar depois, todos os logins anteriores param de valer** — basta logar de novo
`SEED_PROFESSOR_*`	Define o login criado pelo seed. Pode deixar como está
`PORT`	Porta interna do Express. Não mexa; o Compose já mapeia para 3001 na máquina

3.2 Mobile — opcional

Só é necessário para **celular físico**. O app resolve a URL sozinho nos outros casos:

Onde o app roda	URL que ele usa
Emulador Android	`http://10.0.2.2:3001`
Simulador iOS / navegador	`http://localhost:3001`
Celular físico (Expo Go)	precisa de `EXPO_PUBLIC_API_URL`

Para celular físico:

```
cp mobile/.env.example mobile/.env
```

E edite com o IP da sua máquina na rede:

```
EXPO_PUBLIC_API_URL=http://192.168.0.10:3001
```

Descobrir o IP:

```
# Windows
ipconfig
```

```
# macOS / Linux
ipconfig getifaddr en0 || hostname -I
```

> Depois de criar ou alterar `mobile/.env`, **reinicie o Metro com cache limpo**: `npx expo start -c`.
> Variáveis `EXPO_PUBLIC_*` são injetadas no bundle, não lidas em runtime.

---

4 · Subir o banco e a API — escolha **um** caminho

Caminho A — Docker *(recomendado para a entrega)*

Verifique:

```
docker compose version
```

Se não tiver:

```
# Windows
winget install -e --id Docker.DockerDesktop
```

```
# macOS
brew install --cask docker
```

No Windows, o Docker Desktop exige **virtualização habilitada na BIOS** e **reinício** após instalar. Se ele reclamar de WSL 2:

```
wsl --install
wsl --update
```

Com o Docker rodando, na raiz do repositório:

```
docker compose up -d --build
docker compose exec api npm run seed
```

- API: `http://localhost:3001`
- MongoDB: `localhost:27017`
- Os dados sobrevivem a `docker compose down` (volume `mongodb_data`). Para zerar: `docker compose down -v`

Caminho B — MongoDB em memória *(sem instalar nada)*

```
cd backend
npm run dev:memory
```

Sobe a API em `http://localhost:3001` **já populada** com professores, alunos e posts. Os dados existem só enquanto o processo estiver aberto. Ideal para desenvolver e gravar a demonstração. Deixe esse terminal aberto.

Caminho C — MongoDB instalado na máquina

```
# Windows
winget install -e --id MongoDB.Server
```

```
# macOS
brew tap mongodb/brew && brew install mongodb-community && brew services start mongodb-community
```

Ajuste `backend/.env`:

```
MONGODB_URI=mongodb://localhost:27017/blogdb
```

E então:

```
cd backend
npm run seed   # só na primeira vez
npm run dev
```

> Nesse caminho a API sobe na porta `3000` (valor de `PORT`), não `3001`. Ou mude `PORT=3001` no `.env`, ou crie `mobile/.env` com `EXPO_PUBLIC_API_URL=http://localhost:3000`.

Verificação (qualquer caminho)

```
curl http://localhost:3001
```

Esperado:

```
{"message":"Blog Dinâmico API - POSTECH Fase 04","version":"2.0.0","endpoints":
["/auth","/posts","/professores","/alunos"]}
```

E o login:

```
curl -X POST http://localhost:3001/auth/login -H "Content-Type: application/json" -d "
{\"email\":\"professor@postech.com\",\"senha\":\"postech2026\"}"
```

Tem que vir `"success":true` com um `token`. Se vier `401`, o banco está vazio → rode o seed.

---

5 · Rodar o app — escolha **um** caminho

Deixe a API do passo 4 rodando em outro terminal.

Caminho 1 — Emulador Android

Verifique:

```
adb --version
```

Se não existir, instale o Android Studio:

```
# Windows
winget install -e --id Google.AndroidStudio
```

```
# macOS
brew install --cask android-studio
```

No assistente da primeira abertura: escolha **Standard**, aceite as licenças e confirme que vêm marcados **Android SDK**, **Platform-Tools** e **Android Emulator**. Depois, em *More Actions → Virtual Device Manager*, crie um dispositivo (ex.: **Pixel 7**, imagem **API 34**).

Configure as variáveis de ambiente:

```
# Windows (PowerShell, como usuário)
[Environment]::SetEnvironmentVariable('ANDROID_HOME', "$env:LOCALAPPDATA\Android\Sdk", 'User')
$p = [Environment]::GetEnvironmentVariable('Path','User')
[Environment]::SetEnvironmentVariable('Path', "$p;$env:LOCALAPPDATA\Android\Sdk\platform-
tools;$env:LOCALAPPDATA\Android\Sdk\emulator", 'User')
```

```
# macOS / Linux — adicione ao ~/.zshrc ou ~/.bashrc
export ANDROID_HOME=$HOME/Library/Android/sdk     # Linux: $HOME/Android/Sdk
export PATH=$PATH:$ANDROID_HOME/platform-tools:$ANDROID_HOME/emulator
```

Feche e reabra o terminal. Com o emulador aberto:

```
cd mobile
npm run android
```

Caminho 2 — Celular físico com Expo Go

1. Instale o **Expo Go** na Play Store / App Store.

2. Crie `mobile/.env` com o IP da máquina (etapa 3.2).
3. Celular e PC na **mesma rede Wi-Fi**.
4. Libere a porta no firewall, se necessário:

```
# Windows
New-NetFirewallRule -DisplayName "Blog API 3001" -Direction Inbound -LocalPort 3001 -Protocol TCP -
Action Allow
```

5. Rode e leia o QR code:

```
cd mobile
npm start
```

Caminho 3 — Navegador *(verificação rápida)*

```
cd mobile
npm run web
```

Abre em `http://localhost:8081`. Serve para conferir se o app e a API conversam. **Não use para a gravação do vídeo** — a banca espera ver mobile.

---

6 · Checklist final

#	Comando	Resultado esperado
1	`node -v`	`v22.13.0` ou superior
2	`ls backend/.env`	o arquivo existe
3	`cd backend && npm test`	`Tests: 22 passed`
4	`curl http://localhost:3001`	JSON com `"POSTECH Fase 04"`
5	login via curl (etapa 4)	`"success":true` com token
6	app abre e lista 3 posts	lista carregada, sem "Network Error"
7	login no app como professor	aparecem a aba **Admin** e o botão **+**
8	login como aluno	**não** aparecem Admin nem o botão +

Credenciais do seed:

Perfil	E-mail	Senha
Professor	`professor@postech.com`	`postech2026`
Professor	`ana.ribeiro@postech.com`	`postech2026`
Aluno	`bruna.lima@aluno.postech.com`	`postech2026`

7 · Problemas comuns

Sintoma	Causa	Solução
`Network Error` no app	API fora do ar ou URL errada	Abra `http://localhost:3001` no navegador. No emulador Android a URL correta é `10.0.2.2`, não `localhost`
Funciona no navegador, não no emulador	`localhost` dentro do emulador é o **próprio emulador**	O app já usa `10.0.2.2` por padrão. Se você criou `mobile/.env` com `localhost`, apague essa linha
Celular físico não conecta	Expo Go está fora da máquina	`EXPO_PUBLIC_API_URL` com o IP da rede + porta liberada no firewall + mesma Wi-Fi
Alterei `.env` do mobile e nada mudou	`EXPO_PUBLIC_*` entra no bundle	`npx expo start -c`
`401` logo depois de logar	`JWT_SECRET` mudou entre reinícios	Fixe o `JWT_SECRET` no `.env` e faça login de novo
Login sempre falha	Banco vazio	`docker compose exec api npm run seed`, ou use `npm run dev:memory`
Metro trava ou erra pacote	Cache do bundler	`npx expo start -c`
Erro de engine / sintaxe no Metro	Node abaixo de 22.13	Atualize o Node (etapa 1.1)
`EADDRINUSE :3001`	Outra instância da API já está de pé	`docker compose down` ou feche o terminal do `dev:memory`
`docker compose` não existe	Docker não instalado ou não iniciado	Instale e **abra o Docker Desktop** antes dos comandos
`npm ci` falha com lock desatualizado	Lock fora de sincronia	`npm install`

Rota de emergência

Se nada funcionar e você precisa do app no ar **agora**, estes três comandos não dependem de Docker, emulador nem configuração:

```
cd backend && npm ci && npm run dev:memory
```

```
cd mobile && npm ci && npm run web
```

8 · Comandos do dia a dia

Backend

Comando	O que faz
`npm run dev`	API com nodemon, usando o `MONGODB_URI` do `.env`
`npm run dev:memory`	API + MongoDB em memória, já populada
`npm run seed`	Repopula o banco (apaga posts e usuários antes)
`npm test`	Suíte Jest com cobertura
`npm start`	Modo produção

Mobile

Comando	O que faz
`npm start`	Metro + QR code
`npm run android`	Abre no emulador / dispositivo Android
`npm run ios`	Simulador iOS (só macOS)
`npm run web`	Navegador
`npx expo start -c`	Metro com cache limpo

Docker

Comando	O que faz
`docker compose up -d --build`	Sobe MongoDB + API
`docker compose logs -f api`	Logs da API
`docker compose exec api npm run seed`	Popula o banco
`docker compose down`	Para tudo, mantém os dados
`docker compose down -v`	Para tudo e **apaga** os dados

9 · Contexto do projeto

Para colar junto deste arquivo quando pedir ajuda a um assistente em outra máquina.

Stack: React Native 0.88 / Expo SDK 58, React Navigation 7, Axios, Context API no mobile; Node + Express 4, Mongoose 8, JWT, bcryptjs no backend; MongoDB 6; Jest + Supertest + mongodb-memory-server nos testes.

Estrutura:

```
tech-challenge-fase4/
├── backend/src/     app.js · server.js · devServerMemoria.js
│                    config · models (Post, User) · controllers
│                    middlewares (auth, errorHandler) · routes
│                    seeds · utils · tests
├── mobile/src/      api · components · contexts · hooks
│                    navigation · screens · theme · utils
├── docs/            DOCUMENTACAO.md · ROTEIRO-VIDEO.md
├── docker-compose.yml
├── README.md
└── SETUP.md         (este arquivo)
```

Pontos que costumam gerar dúvida:

- Professores e alunos moram na **mesma collection** `users`, separados pelo campo `role`. As rotas `/professores` e `/alunos` saem da mesma fábrica de controller.
- A aba **Admin** do app **não é montada** quando `role !== 'professor'` — não é só esconder botão. A mesma regra é garantida no backend pelo middleware `autorizar('professor')`.
- `UserListScreen` e `UserFormScreen` atendem professores e alunos; o que muda é o parâmetro `role` da rota.
- Leitura de posts é **pública**. Criar, editar e excluir post, e todo o CRUD de usuários, exigem professor autenticado.
- A URL da API é resolvida por plataforma em `mobile/src/api/client.js`.

Documentação completa: `README.md` e `docs/DOCUMENTACAO.md`.